



FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

August 2026

Abstract

A semantic cache answers a new query by returning the stored response to a sufficiently similar older one. Such caches are evaluated on hit rate, latency, and sometimes false-hit rate. None of those asks whether a served hit was still *true* at the moment it was served. I name that quantity *stale-hit-rate*, the fraction of served hits whose content had already expired, and present FreCoS, an extension to GPTCache that fits an exponential validity-decay rate per query cluster and consumes it twice: as a hard validity gate on the serve path, and as a decay term in a cost-aware eviction value function. Cluster identity is recovered by k-means over embeddings from the cache’s own encoder, so both consumers see only signals a deployed system would have. On a seeded synthetic workload with per-cluster content half-lives, the gate cuts stale-hit-rate roughly eightfold against a frequency-based floor, and fitting a rate per cluster beats pooling across clusters with a large, significant effect while staying measurably short of an oracle that reads the true rates. The cost-aware eviction term does not hold up: under real eviction pressure it is indistinguishable from a pure-frequency policy, and plain LRU beats both. False-hit-rate, which the semantic index makes measurable and which is reported beside every result, stays near 0.90 and is the dominant limit on absolute numbers here, though not on relative comparisons.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call, and serves a stored response when an incoming query lands close enough in embedding space to one already answered. The promise: skip the call, return the same answer faster and for free. But “the same answer” is a claim about time as well as similarity, and most semantic caches never check the time part.

This matters most where a language model sits between a user and an answer about content that changes on its own schedule. Monitoring how a brand is represented across AI-generated answers is one such case. The same style of question gets asked repeatedly, exactly the workload a semantic cache absorbs, but the facts behind it do not sit still. A cache that asks only “have I seen a similar question” and never “is that answer still true” keeps serving a stale answer indefinitely once written, and there is no way to tell a correct hit from a wrong one, because stale-hit-rate does not exist in standard cache evaluation.

Two gaps follow, and each motivates one of the components I add. The serve path has no notion of staleness: a cached answer about a fact with its own expiry schedule is served indefinitely, and “found in cache” is silently treated as “correct.” Eviction has no cost term: count-based policies such as LRU and LFU treat two entries with identical access patterns alike even when one is far more expensive to regenerate, so a rarely accessed but costly entry can be evicted ahead of a cheap one accessed once more recently.

Industry and research reflect this gap unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock TTL and no similarity matching. Semantic-caching products use a single fixed global threshold with plain TTL or count-based eviction, with no notion of per-topic staleness. The research literature is further ahead, proposing cost-aware and staleness-aware eviction formulas,

but none names or measures stale-hit-rate. I compare against that literature, not the weaker product baseline.

This report makes three contributions. The first is stale-hit-rate as a named, measured metric, evaluated on a harness that uses a real cosine-similarity index and real k-means clustering rather than exact-match lookup and ground-truth cluster labels, so the gate and the eviction term see only what a deployed cache would see. The second is evidence that per-cluster staleness fitting does measurable work: learned rates beat a pooled rate with a large, significant effect, and stay significantly short of an oracle that reads the generator’s true rates. That is the pattern a fitter working off an imperfect but real signal should produce. The third is a negative result on cost-aware eviction, measured under real eviction pressure: FreCoS’s cost term is indistinguishable from a pure-frequency floor, and plain LRU beats both, for a reason Section 5.3 traces to the workload rather than to any staleness mechanism.

1.2 Related work

The closest prior art, and my primary eviction comparator, is Biton and Friedman’s work on semantic-cache eviction [1]. It proves optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online policies combining recency, frequency, and locality, evaluated on GPTCache with released code, part of why I chose GPTCache here. Two of their findings shape my design: LRU is weak on semantic workloads, and frequency-based policies are a stronger baseline, which is why every comparison in this report treats an LFU-class policy, not LRU, as the floor. FreCoS differs in adding two orthogonal signals, regeneration cost and content validity over time, and in evaluating against a correctness metric their setting never defines.

The structurally closest published formula is Cortex’s LCFU policy [2], scoring an entry as log-scaled frequency times cost times latency times a staleness term, divided by size, with a TTL purge on top. The difference is narrow but load-bearing: Cortex’s staleness term is a static score assigned by a language model at write time, while FreCoS learns a decay rate per cluster from observed staleness. Section 5.1 tests whether that learning does measurable work. I did not reimplement Cortex’s formula, since it relies on remote tool-call latency metadata with no equivalent here.

Several other lines touch individual pieces without matching the combination: GDSF [3] ages access recency, not content validity; Category-Aware caching [4] assigns load-based, not learned, per-category TTLs (which is why I cut a third planned component, calibrated per-cluster thresholds, as a lossy special case of it); FreshCache [5] puts staleness in the serve path as a probabilistic risk budget rather than a hard gate; SCALM [6] clusters entries much as my k-means step does but with no staleness-aware significance score; MeanCache [7] is cost-motivated but does not target eviction; W-TinyLFU [8] is admission control, orthogonal to both my contributions. I considered and rejected vCache [9]: its contribution, per-prompt threshold learning with eviction explicitly out of focus, would have diluted rather than sharpened the comparison.

2 Baseline

2.1 Why GPTCache

I extended GPTCache [10] rather than a more actively maintained semantic cache, for four reasons. First, its most recent release (v0.1.44, August 2024) is frozen, since the maintainers state new integrations have stopped. That makes a better scientific control than a moving target: the baseline measured early in this project is byte-identical to the one measured at the end. Second, architectural fit, verified by reading the vendored source directly: every component I touch sits behind a named interface, so my extension is purely additive (a metadata field via composition, a serving-path gate in one shared decision helper). FreCoS’s eviction policy is the one exception, added by monkeypatching `EvictionBase.get()` rather than through an extension hook, because that factory has none: it is a hard-coded if/elif chain with no lookup table a caller could add to. That path is verified end to end against a real `gptcache.manager.data_manager.SSDataManager` in `tests/test_gptcache_integration.py`. I never edit vendored code. Third, GPTCache is the baseline the closest prior art already uses, letting me in principle run Biton and Friedman’s released policy directly, though in practice I substituted a documented proxy (Section 5.2). Fourth, its default embedding backend runs CPU-deterministically, keeping every run reproducible without a GPU. One trade made deliberately: GPTCache’s README states new contributions are not merged, foreclosing this course’s upstream-adoption bonus.

2.2 What GPTCache does today

GPTCache serves a cached response when a query is semantically close enough to one already seen, using a single global cosine threshold (0.8 by default). Its eviction policies are purely count-based (LRU, LFU, FIFO, random); none reads generation time or regeneration cost, and none retains metadata on an evicted entry. Creation and last-access timestamps *are* persisted, with no downstream consumer. That is the gap my extension repurposes. There is no admission control, and no cost, false-hit, or staleness tracking anywhere in the built-in reporting layer.

3 Design

3.1 One learned artifact, two consumers

FreCoS fits a per-cluster staleness model and uses it at both ends of the cache’s lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. The metric that makes both uses visible is stale-hit-rate: the fraction of served hits whose content was already invalid when served.

Clustering. Cluster assignment comes from a fixed, seeded k-means over cached-query embeddings from GPTCache’s default ONNX model, fit once per trace on the calibration split, with every row assigned to its nearest centroid. Nothing downstream ever reads the generator’s true label; it is recorded only to compute the adjusted Rand index against the learned assignment (`cluster_ari`, reported with every bracket-style result). Both consumers read the same, possibly wrong, cluster identity and neither recomputes it independently. I checked the ablation directly for disagreement between them and found none.

Fitting. For each cluster, a decay rate λ_c is fit from observed staleness in a held-out calibration split, assuming exponential validity decay: $\Pr[\text{still valid after } \Delta t] = e^{-\lambda_c \Delta t}$. A TTL follows at confidence level c as $\text{ttl} = -\ln(c)/\lambda_c$. Three modes produce this table with the same output shape: *learned* fits each cluster independently by maximum likelihood, falling back to a pooled rate below thirty observations; *global* pools every cluster regardless of sample size; *oracle* takes the rate from the generator’s ground truth, an upper bound never available in a real deployment.

Consumer 1: the validity gate. On a candidate hit, if the entry’s age exceeds its cluster’s TTL, it is treated as a miss and the event counts as a stale hit *prevented* rather than served. Age is measured from creation time, never last-access time: staleness is a property of when an answer was generated, not of how recently it was asked for, so a popular but outdated entry is not treated as fresh just because it is popular.

Consumer 2: the eviction value function.

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen_cost}) \cdot e^{-\lambda_c \cdot \text{age}} \quad (1)$$

Here age is again from creation time; $\kappa = 1000$ log-compresses regeneration cost so its heavy tail cannot dominate; the victim is the lowest-value entry, ties broken by oldest creation then lowest entry id. Unlike GDSF-style formulas and Cortex’s LCFU, there is no division by `size_bytes`. Eviction here runs under an entry-count budget, not a byte budget: freeing a 10-byte entry and a 10,000-byte entry both free exactly one slot, so size-normalization has no budget economics to act through. An isolation run at the highest-eviction-pressure cache size in the sweep confirmed this empirically: with and without the term, FreCoS was statistically indistinguishable on every metric (`results/ablation/size_term_isolation/`). Byte-budgeted eviction, where size would carry real meaning, is future work.

The frequency term is $\log(2 + \text{freq})$ rather than the more natural $\log(1 + \text{freq})$, and the difference matters more than it looks: $\log(1 + \text{freq})$ is exactly zero for a never-accessed entry, which zeroes the whole product regardless of cost or freshness and makes a freshly written entry the next victim every time. A property test caught this against the literal formula (Appendix A); $\log(2 + \text{freq})$ has the same concave shape but is strictly positive at zero frequency.

Because the gate already refreshes anything past its TTL, the decay term is not doing staleness prevention on its own. It acts as a soft prior among candidates the gate has already judged fresh. Section 5.2 tests exactly that.

Table 1: Every tunable parameter, its default, and where its effect is measured. **Config** ships $c = 0.95$; the bracketing and ablation experiments run at $c = 0.90$, called out wherever those results appear. Cache size is 412 entries in the bracketing and ablation experiments; the cache-size sweep identifies 248 entries as the knee (Section 5.4). κ and the thirty-observation fallback were fixed at values reasonable at design time and not independently swept.

Parameter	Default	Swept	Measured in
lambda source (mode)	learned	global, learned, oracle	5.1
TTL confidence c	0.95	0.80, 0.90, 0.95, 0.99	5.4
cluster count K	10	5, 10, 20, 50	5.4
cache size (entries)	412	5%–80% of working set	5.4
κ (cost log-compression)	1000	not swept	n/a
calibration fallback threshold	30 obs.	not swept	n/a

4 Experimental setup

4.1 Workload

All experiments run on W1, a deterministic, seeded synthetic workload generator built for this project. Before other implementation work I timeboxed a feasibility spike into a second workload derived from Wikipedia revision history; an automatic sentence-change rule agreed with hand labels only 40% of the time against an 80% go/no-go bar set in advance (`docs/w2-feasibility.md`), so I proceeded with W1 alone.

W1 generates queries across several streams inside each topic cluster: canonical queries establishing a ground-truth answer, near-duplicate paraphrases, novel long-tail queries that never repeat, and time-shifted repeats. Every cluster draws its own half-life and regeneration-cost distribution, and at least one repeat per answer lands past its true expiry, giving the gate something to reject. Each trace splits by arrival time, 30% calibration and 70% evaluation, so no parameter is fit on data it is later scored against.

One workload-design requirement is easy to get wrong, and getting it wrong silently invalidates everything downstream: *cluster identity has to be recoverable from query text alone*, or per-cluster fitting is never actually exercised. A template that varies only in embedded integers across clusters is not recoverable, because a general-purpose embedder scores such text at 0.6–0.9 cosine similarity regardless of true cluster, which puts adjusted Rand index at random-assignment level (0.02–0.06). W1’s canonical text therefore crosses a distinct topic phrase per cluster (51 real-world subjects, e.g. “the Roman Empire”, “quantum computing”) with a distinct aspect and qualifier phrase per answer within that cluster (750 combinations). That puts median `cluster_ari` at 0.42, a substantial signal though not a perfect one. `tests/test_w1.py` asserts it stays above 0.5 on a held-out check.

The default generator draws half-life from an exponential distribution, matching the fitter’s own assumption exactly. Section 5.1.1 reruns the bracketing experiment with that assumption deliberately violated: once with half-life drawn from a Weibull distribution (same mean, different shape), and once, the stronger check, from a mixture of two exponentials within each cluster.

The generator’s two ground-truth distributions, regeneration cost and content half-life, were meant to be calibrated against the Azure LLM Inference Trace and observed update-frequency in a public corpus. Neither was reachable from my development environment, so both are parameterized from commonly cited public figures instead: a plausibility check, not a real external fit, and a limit on external validity.

4.2 Harness and simulation model

The harness is a pure trace replayer; no language model is ever called. On a miss, response content and cost are whatever the trace recorded, and miss latency is drawn from a seeded log-normal scaled by entry size, deterministic per query. That distribution is a documented placeholder, not fit to a real trace. Hit latency and extension overhead (index lookup plus gate check) are measured for real: about 1 ms per decision (Table 5), dominated by the cosine search rather than the gate. Reported latency is therefore

a hybrid of measured and simulated cost: the right trade for keeping runs free and deterministic, but a real limitation.

Lookup uses `benchmarks.semantic_index.SemanticIndex`, brute-force cosine similarity at GPT-Cache’s default 0.8 threshold. Having a real index rather than exact-match lookup is what makes false-hit-rate a live metric, and it is high: median 0.90–0.92 across every bracket-style experiment. Direct inspection of embeddings shows why. Two different aspects of the *same* topic (two distinct questions about Renaissance art, say) score 0.7–0.8 cosine similarity, close enough to the match threshold to cross it, because to a general-purpose embedder they are related text. This is a recognizable failure mode for a real semantic cache rather than a workload artifact, but it does mean every hit-rate number below should be read alongside its false-hit-rate.

The cache starts empty in every run, and the first 10% of the evaluation split warms it and is excluded from every metric.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where serve time exceeds the entry’s true expiry. False-hit-rate is the fraction where the served answer’s identity does not match the query’s. Useful-hit-rate is the fraction of served hits (not of all scored queries) that are both correct and non-stale. It is counted directly by a per-row predicate (`benchmarks.metrics.is_useful_hit`) rather than reconstructed as $(n_{\text{hits}} - n_{\text{stale}} - n_{\text{false}})/n_{\text{hits}}$, because `is_stale_hit` and `is_false_hit` are independent predicates that can both hold of the same hit: the subtraction double-counts the overlap and goes negative once both rates are large.

Cost saved sums regeneration cost over useful hits only: a hit that returned an expired answer, or an answer to a different question, saved no backend call the caller would have accepted. Given how high false-hit-rate runs here, that exclusion matters, and it is what keeps `cost_saved_usd` a measurement rather than an upper bound. Adjusted Rand index (`cluster_ari`) measures how well the learned assignment matches the true labels: 1.0 is a perfect match, 0.0 is what random assignment gives in expectation.

Two primary comparisons are designated up front and reported without correction: learned versus global (does per-cluster learning beat pooling), and gate-on versus the LFU floor (does the gate do measurable work). Every other Mann-Whitney test is secondary, corrected via Holm-Bonferroni across the secondary family, with raw and adjusted p -values reported together (`analysis/multiple_comparisons.py`).

Every configuration ran five independent seeds, each with its own freshly generated 3,000-query trace. Results are medians with 95% CIs from a percentile bootstrap over the five per-seed values (10,000 resamples, fixed seed), with raw hit and stale-hit counts alongside every rate, since a five-seed bootstrap CI is coarser than it looks. Group comparisons use Mann-Whitney U with rank-biserial correlation, $r = 2U_a/(n_1n_2) - 1$, as effect size (`analysis/stats.py`, hand-rolled and unit-tested against known closed-form cases: complete separation gives $r = \pm 1$, identical samples give $r = 0$). At $n = 5$ the smallest achievable two-sided p is about 0.008 and cannot alone distinguish a large effect from a small one, so effect sizes accompany every p -value. A non-significant result is an absence of detected difference, never equivalence; where I claim two conditions behave alike, raw counts back it up rather than a large p -value.

Every experiment ran on the machine recorded in its own `env.json`: an Apple M2 Pro (10 cores), 16 GB RAM, macOS, Python 3.13. It is a shared development machine, not a dedicated benchmarking host, which matters for the peak-RSS figure this report already flags as unreliable.

5 Results

5.1 Bracketing: does learned staleness beat naive pooling?

This experiment tests the core claim behind fitting a rate per cluster at all: does it beat a rate pooled across every cluster, and how far is it from the true rate a real system could never observe? I ran three lambda sources (global, learned, oracle) for 5 seeds each on a 3,000-query W1 configuration, gate enabled at $c = 0.90$, FreCoS eviction throughout. Cluster identity comes from real k-means over query embeddings; the oracle arm alone still reads the true label internally, since its lambda table is keyed by true cluster identity and has no other meaning once clustering is imperfect.

Table 2: Learned tracks between global and oracle, which is the per-cluster fitting claim this experiment was designed to test. The primary comparison (learned vs. global) is significant with a large effect; the secondary comparison against oracle is also significant and in the expected direction (learned worse than the unreachable ceiling). All values are medians over 5 seeds; cost saved is in USD. Median `n_hits` out of 1890 scored queries is 678/635/661 for global/learned/oracle; see `results/brackets/results.csv` for per-seed values.

Lambda source	Stale-hit-rate	95% CI	Cost saved	cluster_ari	False-hit-rate
Global	0.091	(0.085, 0.155)	0.188	0.419	0.898
Learned	0.068	(0.051, 0.087)	0.191	0.419	0.911
Oracle	0.048	(0.042, 0.053)	0.187	0.419	0.914

Learned and global, the primary comparison, separate significantly with a large effect in the expected direction ($U_a = 2.0$, $p \approx 0.028$, $r \approx -0.84$): fitting each cluster’s own rate beats pooling every cluster’s observations into one. Learned and oracle, a secondary comparison, also separate significantly with a large effect ($U_a = 24.0$, $p \approx 0.016$, $r \approx 0.92$), in the direction that per-cluster fitting does not fully close the gap to a rate keyed to true cluster identity. That is the shape this result should have: learned beats pooling without matching an unreachable ceiling.

False-hit-rate stays high across all three arms (median 0.90–0.91), for the same-topic/different-aspect reason described in Section 4.2. It is a caveat on every absolute number here rather than a solved problem, but it applies equally to all three arms, so it does not confound the comparison between them.

Three follow-ups (a scarcer-calibration bracket and the two misspecification checks below) reproduce the same qualitative pattern, with `cluster_ari` in the same 0.40–0.45 range. That rules out calibration sample size and half-life-distribution shape as confounds for the clustering-quality finding.

5.1.1 Misspecification: when the exponential assumption is wrong

The first check drew `W1`’s true half-life from a Weibull distribution instead of exponential (same mean, shape 2); the second, stronger check drew it from a 50/50 mixture of two exponentials within each cluster, biased against the fitter’s single-rate maximum-likelihood estimate.

Table 3: Both checks reproduce the well-specified bracket’s pattern: learned below global (large effect; significant for Weibull, not independently significant for the harder mixture at $n = 5$), learned above oracle (significant, large effect, both checks). Weibull: learned vs. global $U_a = 6.0$, $p \approx 0.175$, $r \approx -0.52$; learned vs. oracle $U_a = 23.0$, $p \approx 0.028$, $r \approx 0.84$. Mixture: learned vs. global $U_a = 4.0$, $p \approx 0.076$, $r \approx -0.68$; learned vs. oracle $U_a = 23.0$, $p \approx 0.028$, $r \approx 0.84$.

Check	Lambda source	Median stale-hit-rate	Median cost saved (USD)	cluster_ari
Weibull	Global	0.033	0.173	0.419
Weibull	Learned	0.024	0.177	0.419
Weibull	Oracle	0.003	0.173	0.419
Mixture	Global	0.189	0.232	0.395
Mixture	Learned	0.154	0.238	0.395
Mixture	Oracle	0.119	0.244	0.395

The adversarial mixture shows the highest absolute stale-hit-rate at every lambda source, as it should: the fitter’s single-rate assumption really is wrong for a two-rate mixture, and it costs something. The relative pattern survives both misspecifications, which makes clustering quality and half-life misspecification separate, additive sources of error. `cluster_ari` is essentially unaffected by how half-life is drawn (0.395–0.419 across all three runs), because cluster identity is fixed entirely by the query-text embedding step, upstream of and independent from the half-life model.

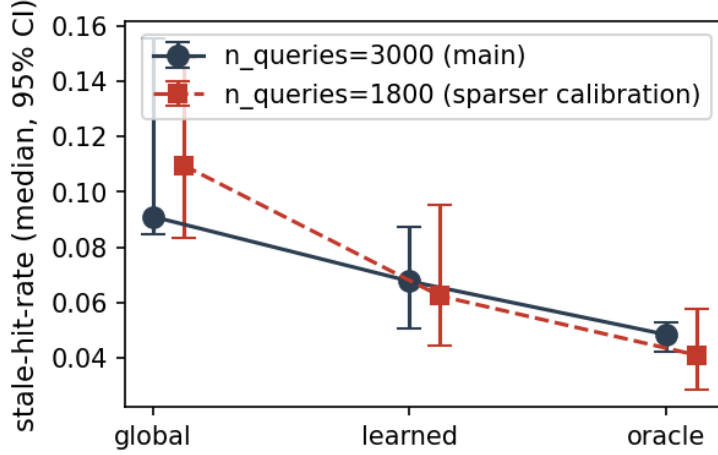


Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the main 3,000-query scale and a roughly 1,800-query sparser-calibration scale. Learned sits between global and oracle at both scales.

5.2 Ablation: isolating the gate from the eviction term

I ran five configurations for 5 seeds each at $c = 0.90$: three gate-disabled eviction policies (LRU, LFU, and a documented substitute for Biton and Friedman’s released policy), then the validity gate combined with plain LFU and with full FreCoS eviction. LFU, not LRU, is the floor of comparison, following Biton and Friedman’s finding that LRU-class policies underperform on semantic workloads.

Table 4: Median stale-hit-rate, hit rate, and false-hit-rate by row, 5 seeds each. Row 2 is the floor. Rows 1–3 are byte-identical seed by seed, and so are rows 4–5; both ties are traced to real causes below.

Row	Configuration	Stale-hit-rate	Hit rate	False-hit-rate
1	LRU, gate off	0.546	0.894	0.923
2	LFU, gate off (floor)	0.546	0.894	0.923
3	B&F substitute, gate off	0.546	0.894	0.923
4	LFU, gate on	0.068	0.336	0.911
5	FreCoS, gate on (full stack)	0.068	0.336	0.911

The gate’s own effect, measured by the second primary comparison of row 4 against the floor, is large and significant: $U_a = 25.0$, $p \approx 0.0090$, $r \approx 1.00$ (perfect separation), cutting stale-hit-rate roughly 8-fold ($\Delta = -0.479$). The gate is a per-entry age check against a per-cluster TTL, and it works whether or not the cluster feeding it is right, because the same assignment is used consistently at calibration and at serve time.

The two ties have different causes, and only one of them is interesting. Rows 1–3 tie because at this configuration’s real semantic hit rate ($\sim 89\%$) only 178–210 misses occur out of 1890 scored queries, under a 412-entry budget. Eviction therefore never runs at all, and the three policies tie because none is ever called to select a victim, not because they agree on one. Section 5.3 confirms this by rerunning the same gate-off configuration at a cache size small enough to force real eviction, where they do differ. Rows 4–5 tie for a substantive reason: FreCoS’s decay term on top of an already-active gate gives $\Delta = 0.000$, byte-identical. The gate accounts for the entire measurable stale-hit-rate improvement, matching the design’s own framing of the decay term as a soft prior: with the gate active, every entry eviction sees is already known-fresh, so the decay term has nothing left to differentiate on.

Table 5: Latency and resource use at the LFU floor (row 2) and the full stack (row 5), medians over 5 seeds. Latency and throughput are dominated by the simulated miss-cost distribution (Section 4.2); only overhead and CPU are measured quantities.

Metric	Row 2 (floor)	Row 5 (full stack)
Extension overhead per decision (ms), measured	1.04	0.82
Latency, mean (ms), simulation-dominated	15.64	95.55
Latency, p95 (ms), simulation-dominated	124.40	280.50
Latency, p99 (ms), simulation-dominated	267.25	419.42
Throughput (queries/s), simulation-dominated	64.0	10.5
Peak RSS (MB), sampled	179.3	183.7
CPU (%), measured	61.3	72.8

Peak RSS lands in the same place in both rows (per-seed 178–183 MB at the floor, 175–194 MB for the full stack), which is unsurprising given both run through the same real index on a shared machine. Extension overhead differs between the two rows in the table, but the per-seed ranges overlap (0.79–1.39 ms at the floor, 0.71–1.12 ms for the full stack), so that difference is run-to-run variation on a shared machine, not an effect of the gate. The mean-latency gap (16 ms to 96 ms) is large because the gate converts a large fraction of hits into misses at this hit rate, and a simulated miss costs far more than a real cache-side decision. The gap is still simulation-dominated, not a claim about real end-to-end latency. I report no latency CDF, though the guideline asks for one, for the same reason: its shape would be the shape of a random-number generator, not of any real system. For the one measured latency quantity, extension overhead, the seed-to-seed spread is the range quoted above.

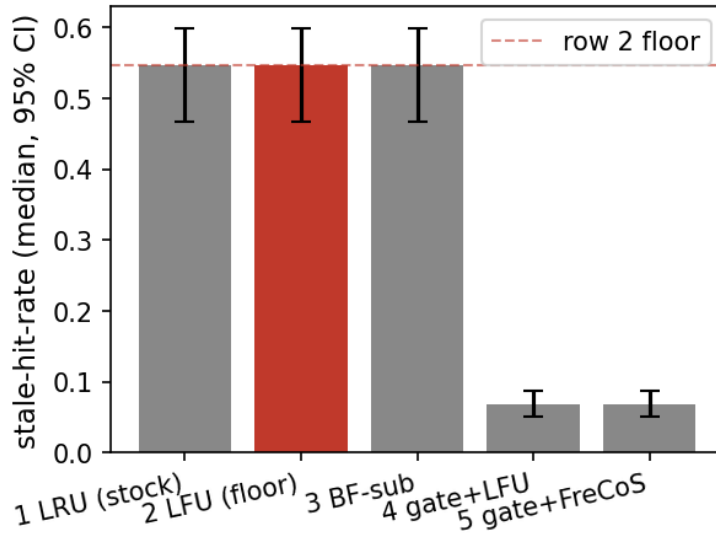


Figure 2: Stale-hit-rate across all five ablation rows, with 95% confidence intervals. Row 2, LFU with the gate disabled, is the floor of comparison.

5.3 A fair test of cost-aware eviction

Every ablation row above ran either with the gate on, which pre-filters stale entries before eviction ever sees them, or at a cache size where eviction never fires. Neither gives FreCoS’s cost and decay terms anything to differentiate on. This experiment turns the gate off and runs three eviction policies for 5 seeds each at a 25-entry cache, scaled down from the 412 entries the ablation uses in proportion to the smaller trace so that the cache reliably fills. It does: 1,325–1,515 misses occur per run out of 1890 scored queries, against 178–210 in the gate-off ablation rows. That is real eviction pressure.

Table 6: With real eviction pressure, FreCoS’s cost term is indistinguishable from the LFU floor on cost saved, and LRU beats both on cost saved and on stale-hit-rate. Cost saved counts useful hits only (Section 4.3), which is why the absolute values are small.

Policy	Median cost saved (USD)	95% CI	Median stale-hit-rate
FreCoS	0.138	(0.084, 0.234)	0.420
LFU	0.120	(0.075, 0.216)	0.526
LRU	0.213	(0.133, 0.365)	0.092

FreCoS versus LFU on cost saved: $U_a = 13.0$, $p \approx 0.917$, $r \approx 0.04$. That is no effect at all. The two medians sit well inside each other’s confidence intervals and the per-seed values interleave. This experiment was built to be the eviction term’s best case, the one setting where the cost and decay factors in Equation 1 have something to differentiate on, and under that setting they do not separate from a plain frequency count.

LRU beats FreCoS on cost saved ($U_a = 5.0$, $p \approx 0.117$, $r \approx -0.60$, large effect, not significant at $n = 5$) and is strictly lower on stale-hit-rate ($U_a = 25.0$, $p \approx 0.0090$, $r \approx 1.00$, perfect separation). It also separates perfectly from the LFU floor on stale-hit-rate ($U_a = 0.0$, $p \approx 0.0090$, $r \approx -1.00$). Useful-hit-rate follows the same ordering (0.123 for LRU, 0.087 for LFU, 0.077 for FreCoS), which is what the cost column is measuring, since cost saved now sums regeneration cost over useful hits only. The mechanism has nothing to do with staleness modeling. Under a 25-entry budget with no gate active, the LRU resident set is close to the 25 most recently touched entries, and since every miss inserts a freshly created entry, an entry that survives under LRU is usually one created recently. Evicting the least recently used entry is therefore very nearly evicting the entry most likely to have expired, and LRU gets that for free: no staleness model, no calibration split, no gate.

I report this as a negative result. FreCoS’s cost term buys nothing measurable over LFU here, and both lose to the cheapest policy in the comparison. Biton and Friedman report LRU-class policies underperforming on semantic workloads, which is why LFU is the floor everywhere else in this report; on this trace the opposite holds. The reason is a property of the trace rather than of the value function: creation age and access recency line up closely enough that recency alone approximates freshness. Separating the value function from recency needs a workload where those two orderings come apart, which W1 does not provide.

5.4 Cache size and the correctness-efficiency trade-off

I swept cache size across 5 points spanning 5%–80% of the workload’s distinct-answer working set, full stack throughout. Hit rate reaches its ceiling at 15% of the working set (248 entries) and flattens completely from there: every point from 248 entries on produces identical per-seed hit counts. The knee is computed from the sweep’s own data by `analysis/fig3_cache_size.py`, which takes the smallest cache size whose median hit rate is within 1% of every larger size’s, rather than from a hardcoded constant, so the figure and its caption cannot drift apart from the CSV. This cache size anchors the remaining sweeps.

Table 7: Useful-hit-rate (correct and non-stale, among served hits only; Section 4.3) rises monotonically with TTL confidence, moving opposite to false-hit-rate, while stale-hit-rate falls to zero over the same range.

TTL conf.	Med. stale-hit-rate	Med. hit rate	Med. useful-hit-rate	Med. false-hit-rate
0.80	0.142	0.486	0.042	0.946
0.90	0.068	0.336	0.080	0.911
0.95	0.046	0.231	0.139	0.857
0.99	0.000	0.079	0.458	0.542

Useful-hit-rate rises from 0.042 at confidence 0.80 to 0.458 at 0.99 (0.95 vs. 0.99: $U_a = 0.0$, $p \approx 0.0090$, $r \approx -1.00$, perfect separation). False-hit-rate falls substantially over the same range, 0.946 to 0.542, and the two movements share one cause: a tighter TTL evicts stale entries faster, which shrinks the pool of cached entries available to match against, and with fewer (mostly correctly clustered) entries surviving, a smaller fraction of hits are false. Tightening the TTL therefore buys correctness on two axes at once, at a steep price in hit rate, which falls from 0.486 to 0.079 across the same range. That trade-off, not either axis alone, is the practical result of this sweep.

A third sweep varied cluster count at 5, 10, 20, and 50. Hit rate and stale-hit-rate stay non-monotone with heavily overlapping CIs, and `cluster_ari` is non-monotone too (0.451 at $K = 5$, 0.272 at $K = 20$, 0.320 at $K = 50$). This is not topic-vocabulary exhaustion: the generator’s 51 distinct topics comfortably cover every K tested, so no cycling occurs. The likelier explanation is that more, smaller ground-truth clusters give k-means proportionally fewer calibration observations per cluster at a fixed trace size. At 5 seeds per point, though, that is a plausible reading of a wide-CI, non-monotone result rather than a confirmed effect. It therefore sits in a supplementary table (`analysis/figures/supplementary.csv`) rather than a headline figure, since it clears neither the effect-size nor the confidence-interval bar the rest of this report’s figures were held to.

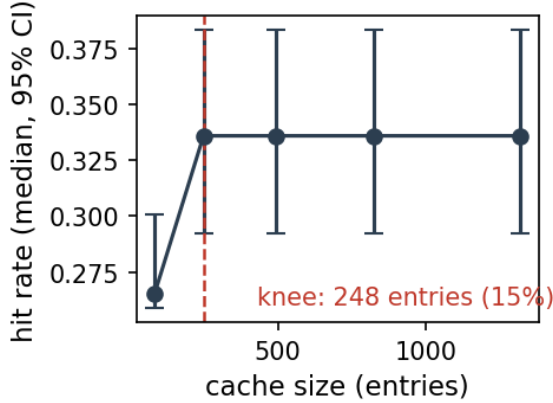


Figure 3: Hit rate against cache size, with the identified knee at 15% of the working set (248 entries) marked.

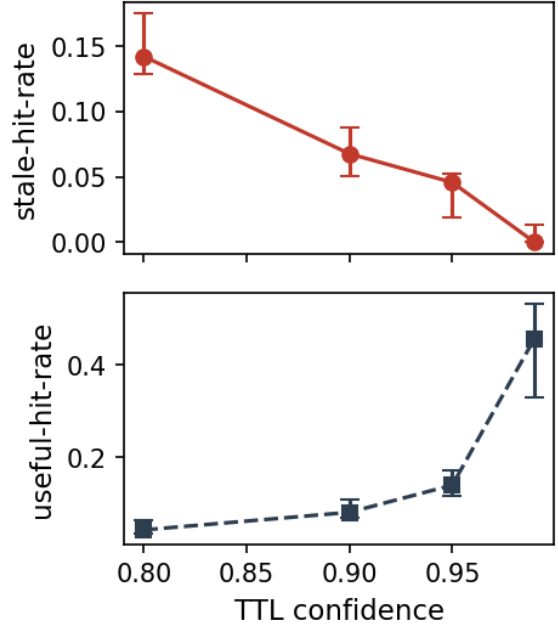


Figure 4: Stale-hit-rate and useful-hit-rate against the gate’s target confidence level. Stale-hit-rate falls to zero; useful-hit-rate rises from 0.04 to 0.46.

6 Discussion

Three findings hold, at very different strengths.

The gate is the strongest of them. It cuts stale-hit-rate roughly 8-fold against the LRU floor with perfect separation across seeds, and it does so *independently of clustering quality*, because the same cluster assignment is used at calibration and at serve time: a wrong but consistent cluster still yields a usable TTL. That robustness is why this component survives every variation tested here.

Per-cluster learning is the more interesting finding. Learned rates beat a pooled rate with a large, significant effect and stay significantly short of oracle, which is what a working fitter should produce given an imperfect signal (median `cluster_ari` 0.42). The result also depends on a workload property that is easy to get wrong: if cluster identity is not recoverable from query text, per-cluster fitting is never exercised at all, and a suite can report a clean-looking null result that measures nothing (Section 4.2). Any future work extending this to a real corpus should verify `cluster_ari` before trusting any per-cluster result.

Cost-aware eviction is the weakest, and it fails. Tested under real eviction pressure, in the setting designed to give it every advantage, FreCoS’s cost term is indistinguishable from the pure-frequency floor, and plain LRU beats both on cost saved and on stale-hit-rate. The reason has nothing to do with modeling staleness and everything to do with access recency tracking creation age on this trace. On workloads with that structure, a policy with no staleness model at all is hard to beat.

Two limitations cut across everything above. First, false-hit-rate near 0.90: the semantic index matches same-topic, different-aspect queries at 0.7–0.8 cosine similarity, close enough to the 0.8 threshold to cross it. This applies uniformly across arms, so relative comparisons stand, but no absolute

number here should be read as production-realistic. Second, circularity: stale-hit-rate is scored against `valid_until`, the same ground-truth field that supplies the oracle arm’s λ . The metric, the ground truth it is checked against, and the ceiling one arm is measured relative to all come from one self-authored generator. Relative comparisons within an experiment remain valid, since both sides are scored against the same ground truth, but no number here is a claim about staleness detection against an independent notion of correctness. My attempt at external ground truth from Wikipedia stopped at a feasibility spike that measured 40% rule agreement against an 80% bar. Neither prior-art comparator, Biton and Friedman’s released policy nor Cortex’s LCFU, was run against this workload, so the eviction axis is measured only against count-based policies and FreCoS itself.

Finally, scale: 3,000 queries and 5 seeds per configuration, chosen because embedding dominates runtime and the conclusions here do not depend on trace size. The cost is statistical power: several comparisons show a large effect and a consistent direction without clearing $p < 0.05$ at $n = 5$ (LRU vs. FreCoS on cost saved, most notably). Every such case is reported with its effect size next to the non-significant p -value rather than as a null result.

7 Conclusion and future work

I set out to name a gap in how semantic caches are evaluated, stale-hit-rate, and to build a system that closes it, with two components matching the two gaps in Section 1.1. The validity gate does the job: an 8-fold cut in stale-hit-rate with perfect separation, robust to clustering error. The per-cluster learning that sharpens it holds up as well, beating pooling with a large, significant effect while staying short of oracle. The eviction term does not: tested under real eviction pressure, its cost-awareness is indistinguishable from a pure-frequency floor, and plain LRU beats both on this trace for reasons unrelated to staleness modeling.

The most consequential next step is the false-hit-rate gap: same-topic, different-aspect queries still land close to the index’s match threshold, so a more semantically distant aspect vocabulary, or a real corpus in place of a synthetic template, would likely reduce false-hit-rate without touching the clustering pipeline. The eviction result points somewhere specific too: building a workload whose creation-age and access-recency orderings disagree, so the value function has something to supply that recency does not already. Beyond that: running Biton and Friedman’s policy and Cortex’s LCFU directly, now that a real index exists to run them against; calibrating the generator against a real inference trace; and breaking the circularity between stale-hit-rate and its generator-supplied ground truth, which would be the single strongest improvement available. Dynamic re-clustering, a FreshCache-style probabilistic staleness budget, byte-budgeted eviction (where the size term absent from Equation 1 would carry real meaning), and admission control remain future work.

Appendix A: correctness evidence

Correctness is checked at four levels. A reference oracle (`tests/oracle/reference_cache.py`) implements the same hit/miss/stale decision as the extension using a naive dict-and-linear-scan approach with no shared code, and every pipeline decision is cross-checked against it in `tests/test_stock_parity.py`. A callable invariant suite (`tests/invariants.py`) runs inside both the unit tests and `benchmarks/harness.py` on every experimental run, checking budget bounds, argmin eviction selection, that no entry is served past its cluster’s TTL, and that staleness decisions derive from creation time and never last-access time. Property-based tests (`tests/test_freco.py`) check that the eviction value function is monotonic in each term independently. Finally, `tests/test_gptcache_integration.py` drives a real `gptcache.manager.data_manager.SSDataManager` through enough writes to force eviction and confirms the victim GPTCache’s own eviction factory selects matches what `FreCoSEviction.select_victim()` computes from the same metadata. FreCoS is therefore demonstrably reachable through GPTCache’s real eviction path, not merely correct in isolation.

The most persuasive of these artifacts is a property test that failed against my original design rather than a bug found by inspection: `test_cold_start_entry_scores_above_zero` asserts a fresh entry must score above zero, and failed against the literal $\log(1 + \text{freq})$ formula. That is how the $\log(1 + \text{freq}) \rightarrow \log(2 + \text{freq})$ correction in Section 3.1 was found.

Appendix B: code and data artifacts

This repository is public at <https://github.com/LiorLotan2/frecos>. A `Dockerfile` and `environment.yml` (or `requirements.txt` for a plain `venv`) build a working environment from a clean clone, and `.github/workflows/ci.yml` reruns the full test suite, a benchmark smoke test, an experiment-path smoke test, and a figure-consistency check on every commit. `benchmarks/samples/` holds committed sample benchmark logs, reproducible with `make bench-smoke`. `make experiments` reruns all seven experiments behind Section 5 in dependency order; `make figures` regenerates every figure from the resulting CSVs. No figure in this report is hand-edited.

Table 8: Where every artifact referenced in this report lives in the repository.

Artifact	Location
Serve-path decision seam	<code>gptcache_ext/pipeline.py</code>
Additive metadata storage	<code>gptcache_ext/metadata.py</code>
Staleness model (fitter, gate)	<code>gptcache_ext/staleness/</code>
Real clustering (embedder, k-means, ARI)	<code>gptcache_ext/staleness/embedder.py</code> , <code>clusters.py</code> , <code>assign_real_clusters.py</code> , <code>cluster_accuracy.py</code>
Eviction (FreCoS and baselines)	<code>gptcache_ext/eviction/</code>
Semantic index (cosine, 0.8 threshold)	<code>benchmarks/semantic_index.py</code>
Synthetic workload generator (W1)	<code>workloads/w1_synthetic/</code>
Harness, metrics, experiment runners	<code>benchmarks/harness.py</code> , <code>metrics.py</code> , <code>runners/</code>
Raw results, summary, environment per experiment	<code>results/*/results.csv</code> , <code>summary.md</code> , <code>env.json</code>
Statistics and multiple-comparison correction	<code>analysis/stats.py</code> , <code>multiple_comparisons.py</code>
Reference oracle, invariants, property tests	<code>tests/oracle/reference_cache.py</code> , <code>tests/invariants.py</code> , <code>tests/test_frecos.py</code>
Real GPTCache eviction-factory integration test	<code>tests/test_gptcache_integration.py</code>
GPTCache pinned commit and source map	<code>docs/baseline-source-map.md</code>
Wikipedia feasibility spike	<code>docs/w2-feasibility.md</code>
Reproduce experiments / figures / report	<code>make experiments</code> , <code>make figures</code> , <code>make report</code>
Environment and CI	<code>Dockerfile</code> , <code>environment.yml</code> , <code>requirements.txt</code> , <code>.github/workflows/ci.yml</code>

References

- [1] D. D. Biton and R. Friedman. *From Exact Hits to Close Enough: Semantic Caching for LLM Embeddings*. arXiv:2603.03301, 2026.
- [2] C. Ruan, C. Bi, K. Zheng, Z. Shi, X. Wan, and J. Li. *Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access for LLM via Semantic-Aware Knowledge Caching*. arXiv:2509.17360, 2025.
- [3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies*. Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.
- [4] C. Wang, X. Liu, Y. Zhu, A. Youssef, P. Nagpurkar, and H. Chen. *Category-Aware Semantic Caching for Heterogeneous LLM Workloads*. arXiv:2510.26835, 2025.
- [5] M. Mansoor, T. Ahmad, and Y.-C. Yoon. *Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs*. arXiv:2607.04281, 2026.
- [6] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*. arXiv:2406.00025, 2024.
- [7] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.
- [8] G. Einziger, R. Friedman, and B. Manes. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.
- [9] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.
- [10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. <https://github.com/zilliztech/GPTCache>, 2024. Version 0.1.44, commit bae7ffef774e762d9d4e60fce70be00011188a6.